



~/playground – cursor

```
$ cursor new playground
```

```
# Building IP product...
```

```
✓ React + Vite + Supabase ready
```

```
✓ Awaiting your first prompt_
```

BUILD WITH AI

Cursor Playbook.

Copy-paste prompts to build The PlayGround
from scratch in 3–4 weeks. RM2–5k total.

TOOL	TIMELINE	COST	OUTCOME
CURSOR	3–4 WEEKS	RM2–5K	PRODUCTION APP



AI builds the code. You ship the product.

Cursor is VS Code with Claude/GPT built in. You describe what you want in plain English, AI writes the code, and the code lives on your computer — you own it, you control it, you deploy it anywhere. Unlike Lovable or Bolt, there's no vendor lock-in and no per-message limits.

For The PlayGround, this is the right tool. You're building something operationally simple (forms, photos, scores) but customer-facing — and you want it to feel *yours*, not templated. Cursor gives you that without 6 months of learning to code.

The math vs. other paths

PATH	TIMELINE	COST	OWNERSHIP	CEILING
Hire dev agency	4–6 months	RM80k+	Theirs unless contract	Their bandwidth
Glide / Airtable	6–8 weeks	RM15–25k	Vendor-locked	Hits limits at scale
Lovable / Bolt	2–4 weeks	RM3–8k	Exportable code	Quality varies
Cursor (you build)	3–4 weeks	RM2–5k	100% yours	Unlimited

THE MINDSET SHIFT

You are not learning to code. You are learning to **describe what you want clearly**. The better your prompts, the better the code. This playbook gives you the prompts. Your job: read the AI's output, test in browser, ask for fixes. Repeat.

What you're actually building.

This is the recommended modern stack — same one Lovable and Bolt use under the hood. All beginner-friendly with great AI support, all generous free tiers, all production-grade.

LAYER	TOOL	PURPOSE	COST
Editor	Cursor	AI-powered VS Code. Where you'll spend 90% of time.	\$20/mo (~RM90)
Framework	Next.js 15 (React)	Full-stack — frontend + API routes in one project.	Free
Language	TypeScript	Catches errors before AI gets confused. Worth it.	Free
Styling	Tailwind CSS	AI writes Tailwind beautifully. Don't fight it.	Free
Database	Supabase	Postgres + auth + storage + realtime. One service.	Free → \$25
Photo Storage	Supabase Storage	Built into Supabase. Auto-CDN. 1GB free.	Included
Auth	Supabase Auth	Email/SMS/Google login. Team code login custom.	Included
Hosting	Vercel	Push to GitHub → live in 30 seconds. Free for hobby.	Free → \$20
Domain	Cloudflare / Namecheap	playground.teambuddy.my	RM50/yr
Total monthly		Production-ready, scales to thousands of users	~RM150/mo

Why this exact stack works with AI

Next.js + Supabase is the most documented combo in 2026 — meaning Claude/GPT have read thousands of working examples. When you ask AI "add Supabase auth to my Next.js app," it produces clean, correct code on the first try. Stray from these defaults and AI starts guessing.

Get your environment running in 90 minutes.

Before AI can build anything, you need the basics installed. This is the one part of the process that isn't 'prompt and ship' — but follow the steps and you'll be coding by lunch.

Step 1 · Install the basics (30 min)

Download and install in this order:

TOOL	WHERE	WHY
Node.js LTS	nodejs.org	Runs the app locally. Pick the LTS version.
Cursor	cursor.com	The AI code editor. \$20/mo Pro plan.
Git	git-scm.com	Version control. Cursor uses it for safety nets.
GitHub account	github.com	Free. Your code's permanent home.
Supabase account	supabase.com	Free. Your database + auth + storage.
Vercel account	vercel.com	Free. Where the app goes live. Login with GitHub.

Step 2 · Create the project (10 min)

Open your terminal (Mac: Terminal app · Windows: PowerShell). Run these commands one by one:

```
Terminal bash

# Make a folder for your project
cd ~/Documents
npx create-next-app@latest playground

# When prompted, choose these answers:
# ✓ TypeScript? → Yes
# ✓ ESLint? → Yes
# ✓ Tailwind CSS? → Yes
# ✓ src/ directory? → Yes
# ✓ App Router? → Yes
# ✓ Turbopack? → Yes
# ✓ Import alias? → No (press Enter)

cd playground
code . # Opens in Cursor
```

Step 3 · Open AI chat in Cursor (5 min)

Press **Cmd + L** (Mac) or **Ctrl + L** (Windows). The AI chat panel opens on the right. Make sure the model picker shows **Claude Sonnet 4** or newer. This is where you'll spend most of your time.

PRO TIP · USE COMPOSER MODE

Press **Cmd + I** (or **Ctrl + I**) to open **Composer** — Cursor's most powerful mode. Unlike chat, Composer edits multiple files at once and runs terminal commands for you. Use it for any prompt longer than a sentence.

Week 1 — Foundation prompts.

Copy these into Cursor's Composer (Cmd+I) one at a time. Wait for AI to finish, check the result in your browser (run `npm run dev`), then move to the next.

Prompt 1.1 · Project scaffolding

PROMPT 1.1 copy ↓

I'm building a mobile-first web app called "The PlayGround" – a live game dashboard for team-building events in Georgetown, Penang. 8 zones, 100 games, 5 museums. Teams complete games, submit photos, unlock zones via passcode.

Set up the project structure:

- Create folders: `src/components`, `src/lib`, `src/types`, `src/data`
- Add Lucide React icons (`npm install lucide-react`)
- Create a color constants file at `src/lib/colors.ts` with these brand colors:
 - coral: #FF5A4E (primary)
 - navy: #0E1F3A
 - lemon: #FFD43B
 - mint: #3DDC97
 - cream: #FFF8EE
 - ink: #1A1A1A
 - plum: #9B5DE5
 - sky: #2EC4F1
 - rose: #FF85A2
- Update `tailwind.config.ts` to use these as theme colors
- Replace `src/app/page.tsx` with a centered "The PlayGround" hero in coral on a cream background, navy heading, mobile-first.
- Make sure mobile viewport meta tag is set in `src/app/layout.tsx`

Prompt 1.2 · Game and zone data

PROMPT 1.2 [copy](#) ↓

Create the game data files:

1. `src/types/index.ts` – TypeScript types for:
 - Zone (id, name, label, color, order, threshold, passcode, emoji, lat, lng)
 - Game (id, title, category, color, zone, points, cost, time, icon, difficulty, lat, lng, description, isMuseum)
 - Team (id, name, number, score, completedGameIds, budget, photoCount, unlockedZoneIds)
 - Museum (id, name, short, zone, color, entry, lat, lng, game)
2. `src/data/zones.ts` – Export an array of 8 zones in this fixed order: Beach Street, Armenian Street, Little India, The Esplanade, Muntri Street, Love Lane, Masjid Kapitan Keling, Leith Street. Each zone has order 1-8, threshold (3 or 4), 4-digit passcode, emoji, color from the palette, and Georgetown lat/lng.
3. `src/data/games.ts` – Export 100 games across these categories: STREET HUNT (15), TASTE LAB (15), HERITAGE (12), CREATIVE (12), ESCAPE LOGIC (10), SOCIAL (10), ARENA (10), ART OF MAKING (8), WILDCARD (3), MUSEUM (5). Distribute geographically across zones.
4. `src/data/museums.ts` – 5 museums: Penang History Gallery, Straits Oriental Museum, Pinang Peranakan Mansion, Cheah Kongsii, Khoo Kongsii. Each with a dedicated museum game (60-90 pts).

Prompt 1.3 · Mobile shell + bottom nav

PROMPT 1.3 [copy](#) ↓

Build the app shell as a mobile-first layout (max-width 440px, centered).

1. `src/components/AppShell.tsx` – wraps the whole app. Cream background. Navy body background outside the 440px container.
2. `src/components/Header.tsx` – fixed top header with navy background. Shows team number, team name, current zone (with emoji), live indicator, 3-stat grid (Score in lemon, Rank in coral, Budget in mint), and a coral countdown bar at the bottom showing time left in HH:MM:SS format. Include decorative coral/lemon/mint circles in the background.
3. `src/components/BottomNav.tsx` – fixed bottom nav, 5 tabs: GAMES (Target icon), PATH (KeyRound icon), MAP (Map icon), RANK (Trophy icon), TEAM (Users icon). Active tab background is coral, text white. Inactive text is white at 60% opacity. Navy background, 3px coral top border.
4. Use Lucide icons throughout. Bold "neo-brutalist" style: thick 2.5px black borders, hard 4px black drop shadows on cards, no soft shadows, no gradients on UI. Wrap content in client components ("use client").

Week 2 — Core features.

These prompts build the heart of the app: viewing games, submitting photos, and the zone unlock system. After this week, you'll have a working demo.

Prompt 2.1 · Games list view

PROMPT 2.1 [copy](#) ↓

Build the Games tab. Show a scrollable list of mission cards.

Features:

- Two filter bars at top: zone filter (horizontal scroll, navy background) and category filter (horizontal scroll, cream background, pill buttons)
- Each mission card has: top color band with category name, icon in a box, title with id number, zone + time meta, and a bottom row with POINTS (coral), COST (mint if free), and a PLAY button (navy bg, lemon text)
- Completed missions show as faded with a green "+points" badge top-right
- Locked-zone missions render differently: dashed border, lock icon, zone name, and an UNLOCK button if the zone is "ready"
- Wildcard missions get a "BONUS" flame badge, museum games get a star badge
- All borders 2.5px black, all shadows 4px hard black drop shadow

Prompt 2.2 · Path view (zone journey)

PROMPT 2.2 [copy](#) ↓

Build the Path tab – a vertical journey through all 8 zones.

Features:

- Each zone is a card with a 56x56 circle on the left (zone emoji + order number) connected by a dashed vertical line to the next zone
- Three states per zone:
 1. UNLOCKED – colored background, mint "OPEN" badge
 2. READY – yellow lemon background pulse, "ENTER PASSCODE" button
 3. LOCKED – gray dashed card, lock icon, "Complete N games in [prev zone]"
- Show progress bar per unlocked zone: "X/Y games · need Z to unlock next"
- Include a header card explaining the 8-zone path with a brief description
- Tap a "ready" zone → opens the passcode entry modal

Prompt 2.3 · Passcode unlock modal

PROMPT 2.3 [copy](#) ↓

Build a passcode entry modal for unlocking the next zone.

Features:

- Full-screen overlay with dark navy backdrop
- Modal card with the zone's color as the header band
- Header shows: "UNLOCK ZONE N/8", emoji, zone name, zone label
- Green mint confirmation chip: "X games done in [prev zone] · threshold met"
- 4 OTP-style input boxes for the 4-digit code (52x60px each, auto-advance on type, backspace to previous)
- "UNLOCK ZONE" button: navy background, lemon text, disabled until 4 digits
- On wrong code: red shake animation, "Wrong code. Try again."
- On correct code: closes modal, triggers a full-screen zone unlocked celebration with the zone emoji and "■ ZONE N UNLOCKED ■" message
- Below the inputs, show a demo passcode hint in a yellow pill (for testing)

Prompt 2.4 · Photo upload modal

PROMPT 2.4 [copy](#) ↓

Build a bottom-sheet modal for submitting mission photos.

Features:

- Slides up from bottom on a dark navy backdrop
- Header band uses the mission's color, shows category and title
- Mission info card: icon, description, points badge (navy bg, lemon text), cost badge, zone badge
- Two options:
 1. TAKE PHOTO button – opens device camera via
`<input type="file" accept="image/*" capture="environment">`
 2. USE DEMO button – generates a placeholder image via canvas with the mission icon and title overlaid on a gradient
- After photo selected, show preview with X button to retake
- Auto-resize uploads to max 400px on longest side, JPEG 70% quality, to keep storage costs low
- SUBMIT button: full width, coral bg, white text, "SUBMIT FOR +N POINTS"
- On submit: closes modal, fires a celebration overlay showing "MISSION COMPLETE" + mission title + "+N pts" on a lemon card

Week 3 — Wire up Supabase.

This is where the app stops being a demo and becomes real. Teams' progress saves to Supabase, photos upload to Supabase Storage, and the leaderboard updates live across all devices.

Prompt 3.1 · Database schema

PROMPT 3.1 [copy](#) ↓

I have a Supabase project ready. Help me design and create the database.

Tables needed (write the SQL I should paste into Supabase SQL editor):

1. events – id, name, starts_at, ends_at, status (draft/live/finished), timer_duration_seconds, created_at
2. teams – id, event_id (fk), name, team_number, login_code (6-char), score, budget_remaining, photo_count, war_cry, created_at
3. team_zones – id, team_id (fk), zone_id (text), unlocked_at (timestamp)
4. completions – id, team_id (fk), game_id (int), submitted_at, photo_url, points_awarded, status (pending/approved/rejected)
5. broadcasts – id, event_id (fk), message, sent_by, sent_at

Also:

- Row-Level Security policies: teams can only read/update their own row (matched by login_code stored in their session)
- A view "live_leaderboard" that joins teams ordered by score DESC
- Realtime publication enabled on teams + completions + broadcasts

Generate the full SQL migration. I will run it in the Supabase SQL editor.

Prompt 3.2 · Supabase client + auth

PROMPT 3.2 [copy](#) ↓

Wire up Supabase to the Next.js app.

1. npm install @supabase/supabase-js @supabase/ssr
2. Create src/lib/supabase/client.ts (browser client) and src/lib/supabase/server.ts (server-side client) using @supabase/ssr
3. Create .env.local with:


```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
(leave blank – I'll paste my keys)
```
4. Build a login page at src/app/login/page.tsx:
 - Single input for 6-character team code (uppercase, auto-format)
 - Validates against teams.login_code in Supabase
 - On success, stores team_id in a httpOnly cookie via a Next.js server action
 - Redirects to /play
5. Create middleware.ts to protect /play/* routes – if no team_id cookie, redirect to /login

Prompt 3.3 · Photo upload to storage

PROMPT 3.3 [copy](#) ↓

Replace the demo photo logic in PhotoUploadModal with real Supabase Storage.

1. In Supabase, create a public bucket called "submissions" (50 MB limit per file, image/* only)
2. Add a server action at src/app/actions/submit.ts:

```
export async function submitMission(formData: FormData) {
  - Verify team_id from cookie
  - Upload photo to submissions bucket with path
    events/{event_id}/{team_id}/{game_id}-{timestamp}.jpg
  - Insert into completions table: team_id, game_id, photo_url,
    points_awarded, status='pending'
  - Update team: score += points, budget -= cost, photo_count += 1
  - Use a transaction so all happens atomically
  - Return { success: true, newScore } or { error }
}
```
3. Connect the modal's submit button to call this action
4. After success, refresh the page data using router.refresh()

Prompt 3.4 · Live leaderboard with realtime

PROMPT 3.4 [copy](#) ↓

Build a live-updating leaderboard using Supabase Realtime.

1. Create src/components/Leaderboard.tsx as a client component
2. On mount, fetch all teams for this event ordered by score DESC
3. Subscribe to the teams table changes (postgres_changes event):
 - When any team's score updates, refetch and re-sort
 - Animate position changes with a smooth slide (use framer-motion; npm install framer-motion)
4. Render each team as a card with rank, name, team number, score, completed count. Crown emoji for #1. Highlight "YOU" row in coral.
5. Show a small toast at the top "STORM RIDERS just scored +50!" when another team submits (use sonner; npm install sonner)

Week 4 — Admin panel + go live.

Prompt 4.1 · GM admin panel

PROMPT 4.1 [copy](#) ↓

Build the Game Master admin panel at /admin (separate from team app).

Features:

- Separate auth: GMs log in with email + password (Supabase Auth)
- Dashboard sections:
 1. EVENT CONTROL: start event, pause, end event, reset, broadcast message
 2. PHOTO REVIEW: grid of pending photo submissions. Each card shows team #, mission, thumbnail. Approve (✓) or Reject (✗) buttons. Approve = keep points. Reject = subtract points, refund cost, mark submission as rejected.
 3. LIVE STANDINGS: full leaderboard with raw numbers, plus a chart of score-over-time per team (recharts)
 4. ZONE STATUS: how many teams are in each zone right now
 5. MANUAL ADJUST: search team, edit score/budget/zones manually
- Build as Next.js app routes under src/app/admin/
- Protect with middleware: only authenticated GM emails (whitelist in env var GM_EMAILS=" comma,separated,list ") can access
- Mobile-friendly but optimized for tablet portrait orientation

Prompt 4.2 · Map view with mission pins

PROMPT 4.2 [copy](#) ↓

Build the Map tab.

Features:

- Stylized SVG map of Georgetown (380 x 480 viewBox)
- Show all 8 zones as colored circles, with order numbers
- Show 100 mission pins (white circles with emoji) at game coordinates
- Mission pins for locked zones are HIDDEN
- Done missions show a mint checkmark; museum games show a lemon star
- Tap a pin → opens the same PhotoUploadModal
- Below the map: 8 zone chip cards in a 2-column grid, showing each zone's progress (X/Y games · N%)
- Compass rose in bottom-right
- Add a dashed polyline connecting all 8 zones in path order

Prompt 4.3 · Deploy to Vercel

PROMPT 4.3 [copy](#) ↓

Deploy to production.

1. Initialize git, commit everything, push to a new GitHub repo
2. On vercel.com:
 - Import the repo
 - Add env vars: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, GM_EMAILS
 - Deploy
3. Add the production URL to Supabase: Project Settings → URL Configuration → Site URL = https://playground.teambuddy.my
4. Set up custom domain in Vercel: playground.teambuddy.my
5. Generate a production seed script at scripts/seed-event.ts that creates a sample event with 25 teams (6-char login codes), all 100 games, 8 zones with passcodes. Run with: `tsx scripts/seed-event.ts`
6. Tell me exactly which DNS records to add at my domain registrar.

Prompt 4.4 · QR code generator for zones

PROMPT 4.4 [copy](#) ↓

Add a printable QR poster generator to the admin panel.

For each of the 8 zones, generate an A3 printable PDF poster:

- Big zone emoji at top
- "ZONE N/8" eyebrow
- Zone name (huge, bold)
- Zone label / subtitle
- A 4-digit passcode in a yellow lemon pill, 120pt font
- "ENTER THIS CODE IN THE APP" instruction
- Optional: a QR code that pre-fills the passcode if scanned in the app (encode as `playground.teambuddy.my/unlock?z=zoneId&c=passcode`)
- TheBranded Playground footer

Use react-pdf (npm install @react-pdf/renderer) to generate. Admin clicks "Print zone posters" → downloads a single PDF with all 8 zones.

When AI gets stuck — say one of these.

AI will get something wrong. That's normal. These prompts unstick 90% of issues.

IF A PAGE WON'T LOAD

I'm getting an error when I visit /play. The terminal shows: [paste full error from terminal here]. Look

IF THE UI LOOKS WRONG

The [Header / Modal / Card] doesn't match my brand. It should be: 2.5px black borders, 4px hard black dr

IF AI MADE UP AN API

You used a function/import that doesn't exist. Check the actual Next.js 15 docs and Supabase JS docs bef

IF TYPES ARE BROKEN

TypeScript is complaining: [paste error]. Don't use 'any'. Fix the actual type — usually by checking what

IF SOMETHING STOPS WORKING AFTER A PROMPT

After your last change, [feature X] broke. Without rewriting unrelated code, find what your previous edit

IF YOU WANT A SMALLER CHANGE

Don't refactor everything. I just want [one specific thing]. Touch the minimum number of files. Show me t

IF AI IS GOING IN CIRCLES

Stop. Read the entire file at [path/to/file.tsx] from top to bottom. Tell me what it does in 3 sentences.

THE GIT SAFETY NET

Cursor has built-in version control. After every working feature, run **git add . && git commit -m "working: [feature]"** in Cursor's terminal. If AI breaks something, run **git reset --hard HEAD** to roll back to the last good state. Commit often. Trust nothing.

Your 4-week ship plan.

WEEK 1 · FOUNDATION

- Day 1** Install everything (Cursor, Node, Supabase account, etc). Run prompts 1.1–1.3.
- Day 2** Add zones, games, museums data (prompt 1.2). Verify all 100 games and 8 zones.
- Day 3** Build the app shell, header, bottom nav. Make it look right on your phone.
- Day 4** Polish typography, colors, spacing. Make the brand feel premium.
- Day 5–7** Buffer. Tweak. Commit progress. Take a breath.

WEEK 2 · CORE FEATURES

- Day 8** Games list view (prompt 2.1). Filter by zone + category.
- Day 9** Path view + zone unlock cards (prompt 2.2).
- Day 10** Passcode entry modal (prompt 2.3). Test all 8 codes.
- Day 11** Photo upload modal (prompt 2.4) with demo photos.
- Day 12** Celebration overlays. Animations. Make submission feel rewarding.
- Day 13–14** End-to-end demo: log in → unlock → play → unlock next. No backend yet.

WEEK 3 · BACKEND LIVE

- Day 15** Supabase schema (prompt 3.1). Run SQL. Seed sample data.
- Day 16** Team login + auth (prompt 3.2). Test with 3 demo teams.
- Day 17** Real photo uploads + Storage (prompt 3.3).
- Day 18** Realtime leaderboard (prompt 3.4).
- Day 19** Test on 3 phones simultaneously. Verify they see each other's photos.
- Day 20–21** Buffer for bug fixing. Commit. Test edge cases.

WEEK 4 · ADMIN + LAUNCH

- Day 22** GM admin panel (prompt 4.1). Photo approval flow.
- Day 23** Map view (prompt 4.2).
- Day 24** Deploy to Vercel (prompt 4.3). Set up custom domain.
- Day 25** QR poster generator (prompt 4.4). Print all 8 zone posters.

-
- Day 26** Internal dry-run: 5 friends, 90 minutes, 3 zones. Find everything broken.
-
- Day 27** Fix the broken stuff.
-
- Day 28** Ready. Run a pilot event.

What this actually costs.

LINE ITEM	AMOUNT	NOTES
Cursor Pro (4 months)	RM360	\$20/mo. Cancel after launch.
Domain (1 year)	RM60	Namecheap or Cloudflare.
Supabase Pro (when you outgrow free)	RM110	Only needed if free tier breaches.
Vercel (free tier covers MVP)	RM0	Upgrade to Pro only at scale.
Trademark MyIPO Class 41	RM900	Optional but recommended.
Logo + brand kit (Fiverr)	RM400	Or DIY in Canva for RM0.
Poster design + print (8 × A3)	RM250	Local Penang printer.
Mission cards print (25 sets)	RM450	Optional — app handles this.
Pilot event materials + props	RM800	Budget envelopes, wristbands, etc.
Contingency	RM670	Always something.
TOTAL	RM4,000	Full IP product, live and trademark

REALITY CHECK

Compared to the RM12k–RM35k low-code path or RM80k+ agency build, the Cursor path costs **RM4,000 total**. The difference goes into your pocket — or into marketing, prizes, and a better pilot event. Your first paying client returns 5–10× this investment.

10 things to remember while you build.

01 Prompt small. Test often.

One prompt = one feature. Don't ask AI to 'build the whole app.' Build it brick by brick, run npm run dev between each, commit when it works.

02 Commit after every win.

git commit -m 'working: photo upload' is your safety net. AI is occasionally wrong. Git lets you roll back instantly.

03 Show, don't tell.

Paste screenshots into Cursor's chat. AI sees images. Showing a broken layout is 10× faster than describing it.

04 Be specific about brand.

Generic prompts get generic UI. Repeat your brand colors, border thickness, and shadow style in every visual prompt.

05 Use Composer for big changes.

Cmd+I (Composer) edits across multiple files. Use it for full features. Use chat (Cmd+L) for questions.

06 Read what AI writes.

Don't blindly accept. Skim the code. If you see something weird, ask 'why this approach?' Don't be polite, be curious.

07 AI lies confidently sometimes.

If a function call looks suspicious, paste it back: 'Does Supabase JS v2 actually have this method? Check the docs.'

08 Backend last.

Build all UI with mock data first. Only wire to Supabase once the experience feels right. Switching is fast.

09 Mobile-first means tested on mobile.

Open your dev URL on your actual phone (use ngrok or Vercel preview deploys). Don't trust browser DevTools alone.

10

Ship ugly before polished.

A working ugly v1 with real Supabase data beats a beautiful UI with fake JSON every time. Customers see the value, not the shadow weights.

Do this in the next hour.

Reading playbooks doesn't ship products. Below is the 5-step kickoff you can complete in 60 minutes before you go to bed tonight.

01

Sign up for Cursor

Go to cursor.com. Sign up. Start the free trial. (10 min)

02

Install Node.js

Go to nodejs.org. Download LTS. Install. (10 min)

03

Create a Supabase project

Go to supabase.com. New project: 'playground-dev'. Wait for it to provision. Note the URL + anon key. (15 min)

04

Run prompt 1.1

Open Cursor → new project → paste Prompt 1.1 into Composer (Cmd+I). Let AI scaffold. Run `npm run dev`. See your hero on `localhost:3000`. (20 min)

05

Take a screenshot. Text it to one friend.

Caption: 'Building The PlayGround.' Public commitment is rocket fuel. Tomorrow you'll be on prompt 1.2. By Sunday, you'll have a working app shell. (5 min)

AI WROTE

the code. You shipped the product.

Cursor doesn't make you a developer. It makes you a builder. The skill that matters now isn't typing code — it's knowing what to build, in what order, and when to ship.

Four weeks from now, you'll have a live IP product on your own domain, with real users, earning real revenue. The path is here. The prompts are above. Open Cursor.

TEAMBUDDY · THE PLAYGROUND

Built with AI. Owned by you.